

Modificar el tiempo de expiración del JWT y analizar su efecto.

Objetivo

Modificar el tiempo de vida del token (TTL – Time To Live) para:

- Verificar que el token expira correctamente.
- Observar el comportamiento del sistema cuando el token está vencido.
- Analizar la respuesta de Spring Security ante un JWT expirado.

```
return ResponseEntity.status(status: 401).body(Map.of(k1: "error", v1: "invalid_c\n});\n\nInstant now = Instant.now();\nlong ttl = props.tokenTtlSeconds() != null ? props.tokenTtlSeconds() : 3600;\nInstant exp = now.plusSeconds(ttl);\n\nString scope;\nif (props.username().equals("admin")) {
```

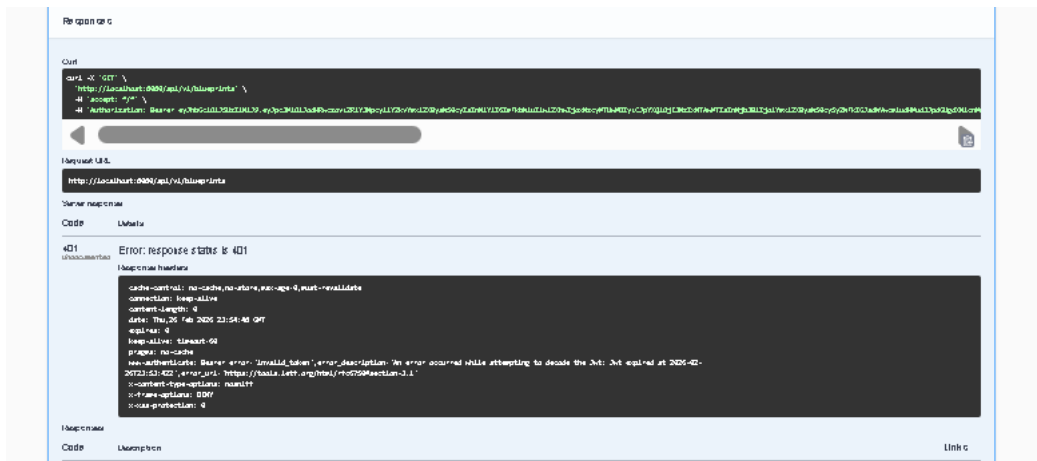
Aquí define el tiempo que tiene de vigencia el token. Eso significa que el tiempo de expiración depende de: `RsaKeyProperties.tokenTtlSeconds()`

```
show-details: always\n\nblueprints:\n  security:\n    issuer: "https://decsis-eci/blueprints"\n    token-ttl-seconds: 10
```

Con el fin de analizar el comportamiento del sistema ante tokens expirados, se modificó el tiempo de vida del JWT mediante la propiedad:

Esta configuración reduce la validez del token a 10 segundos desde su emisión.

Al usar el token se antes de los 10 segundos



Lo cual confirma que Spring Security valida correctamente la claim exp del JWT y rechaza tokens vencidos.

Este comportamiento garantiza que el sistema no acepte credenciales caducadas, fortaleciendo la seguridad ante posibles ataques de replay o robo de tokens.